Taller de Desempeño II – Concurrencia y Caché (RECAUDO-T)

Sep 30, 2026 · @Bryan

Resumen

Con 8 trabajadores el throughput del resumen mensual subió de 0.67 a 24.99 req/s, y al agregar caché la laten cia promedio bajó a 39 ms con 94 % de respuestas servidas desde memoria.

El servidor (`server.py` , Python + Flask) implementa las dos tácticas del taller:

- **R1 Introducir concurrencia:** cada consulta que no está en caché se entrega a un `ThreadPoolExecutor` de W hilos (variable de entorno `WORKERS`). Las solicitudes extra esperan en la cola del pool y no se pierden.
- **R2 Caché:** diccionario en memoria, protegido con `Lock` , con clave `(id_tarjeta, viajes)` . La ruta consulta el caché antes de usar el pool: un HIT responde `desde_cache=true` de inmediato, y un MISS se calcula, se guarda y responde `desde_cache=false` .
- **Cálculo costoso:** duerme entre 150 y 250 ms (uniforme) por cada resumen, igual en todas las corridas.

Para levantar todo con un solo comando:

```
pip install -r requirements.txt
python run_experimentos.py
```

Ese comando levanta el servidor para cada experimento, lanza el cliente, guarda los CSV y al final imprime las métricas.

Configuración de los experimentos

Los tres experimentos usan el mismo cliente y cambian solo `WORKERS` y `CACHE_ENABLED` ; E2 se ejecutó dos veces para confirmar los valores.

Experimento	WORKERS	CACHE_ENABLED	CSV
E0 – línea base	1	false	`e0.csv`
E1 – concurrencia	8	false	`e1.csv`
E2 – concurrencia + caché (1.ª corrida)	8	true	`e2.csv`
E2 – concurrencia + caché (2.ª corrida)	8	true	`e2b.csv`

Parámetros del cliente (`client.py`), iguales en las cuatro corridas:

- Ráfaga de 18 s con 30 hilos que simulan usuarios, reutilizando 10 tarjetas (`1000` a `1009`), 40 viajes por consulta.
- Cada usuario espera entre 0.5 y 1.5 s antes de su siguiente consulta.
- Una solicitud sin respuesta en 3 s cuenta como no procesada (`exito=0`).
- Cada intento queda registrado en el CSV con `timestamp_envio` , `timestamp_respuesta` , `exito` , `desde_cache` e `id_tarjeta` .

Tabla de métricas

E1 multiplica por 37 el throughput de E0 y elimina las solicitudes no procesadas; E2 reduce además la latencia promedio a 39 ms. Cada columna sale del CSV indicado, procesado con `python analyze.py e0.csv e1.csv e2.csv e2b.csv` .

Métrica	E0 (W=1, sin caché) `e0.csv`	E1 (W=8, sin caché) `e1.csv`	E2 (W=8 + caché) `e2.csv`	E2, 2.ª corrida `e2b.csv`
Solicitudes totales	156	456	532	527
Latencia promedio	1527.5 ms	240.1 ms	39.2 ms	39.8 ms
Latencia p95	2864.0 ms	372.3 ms	232.2 ms	206.5 ms
Throughput	0.67 req/s	24.99 req/s	29.60 req/s	29.40 req/s
Jitter promedio	822.9 ms	42.0 ms	13.8 ms	14.9 ms
Eventos no procesados (> 3 s)	91.0 %	0.0 %	0.0 %	0.0 %

Métrica	E0 (W=1, sin caché) `e0.csv`	E1 (W=8, sin caché) `e1.csv`	E2 (W=8 + caché) `e2.csv`	E2, 2.ª corrida `e2b.csv`
Respuestas desde caché	0 %	0 %	94.4 %	94.3 %
Latencia prom. de respuestas desde caché	—	—	12.6 ms	14.0 ms
Latencia prom. de respuestas calculadas	1527.5 ms	240.1 ms	484.0 ms	468.5 ms

Cómo leer E0 y E2:

- En E0 solo 14 de 156 solicitudes respondieron en menos de 3 s, así que su latencia promedio y su p95 se calculan sobre esas 14. El throughput es bajo porque el servidor sigue calculando solicitudes que el cliente ya abandonó por timeout, y la cola nunca se vacía.
- En E2 las respuestas calculadas son las primeras de cada una de las 10 tarjetas: llegan casi a la vez, hacen cola, y por eso tardan más (≈ 470–480 ms) que un cálculo aislado.
- En E2 la latencia p95 (232 ms y 207 ms) queda por debajo del objetivo de 300 ms del escenario de calidad.

Análisis

1. **Throughput de E0 y E1.** Pasó de 0.67 a 24.99 req/s. El throughput no depende de cuánto tarda una solicitud sino de cuántas se procesan a la vez: con W=1 el servidor atiende una por una y la cola crece, y con W=8 hay ocho cálculos de 150–250 ms corriendo en paralelo. Cada cálculo tarda lo mismo, pero en el mismo tiempo se completan muchos más.
2. **Latencia en E1.** La latencia promedio medida por el cliente bajó de 1527 ms a 240 ms, pero lo que bajó fue la espera en cola, no el cálculo, que sigue tardando 150–250 ms. Son dos efectos: la concurrencia no acelera una solicitud ya en ejecución, pero evita que las demás esperen detrás de ella. Además, el 1527 ms de E0 sale solo de las 14 solicitudes que respondieron a tiempo, así que subestima el problema real.

3. **Caché en E2.** El 94.4 % de las respuestas vino del caché en `e2.csv` y el 94.3 % en `e2b.csv`; las de caché tardaron unos 13 ms y las calculadas unos 470–480 ms. Lo medimos con 1000 tarjetas distintas (E2 con W=8 y caché, 463 solicitudes, e2_1000tarjetas.csv): solo el 16.0 % vino del caché contra 94.4 % con 10 tarjetas, y la latencia promedio subió de 39 ms a 200 ms (p95 294 ms; las respuestas de caché tardaron 10.5 ms y las calculadas 235.8 ms). La teoría daba cerca de 22 % de aciertos con unas 530 solicitudes, algo por encima de lo medido. El caché seguiría siendo correcto, pero ayudaría mucho menos, porque solo acelera las consultas repetidas; para que siguiera siendo útil haría falta un tráfico más largo y, con tantas entradas, una política para sacar las menos usadas.
4. **Límite de aumentar trabajadores.** Lo detendrían los recursos de la máquina: los núcleos de CPU (con más hilos que núcleos compiten por procesador en vez de avanzar en paralelo), la memoria que consume cada hilo y, si hubiera base de datos, su límite de conexiones. Ahí la táctica "Introducir concurrencia" deja de alcanzar y hay que pasar a **"Aumentar los recursos"**: más máquinas o más CPU y memoria, no solo más hilos en la misma.
5. **Viaje nuevo después de cachear.** El caché seguiría devolviendo el resumen viejo, sin el viaje nuevo, porque nada le avisa que el dato cambió: aparecen datos obsoletos. Hay que agregar invalidación o expiración: un tiempo de vida (TTL) por entrada, o mejor, borrar la entrada de esa tarjeta en el momento en que se registra un viaje, para que la siguiente consulta recalcule.

Archivos entregados y verificación

La entrega incluye el código, los cinco CSV y los logs del servidor, y cumple las tres verificaciones sugeridas por el enunciado.

Archivo	Contenido
`server.py`	Servidor con pool de W hilos y caché en memoria (`/resumen/{id}` y `/config`)
`client.py`	Cliente de carga que escribe el CSV de cada corrida
`analyze.py`	Cálculo de latencia, throughput, jitter, no procesados y % de caché

Archivo	Contenido
`run_experimentos.py`, `run_experimentos.sh`	Ejecución de E0, E1 y E2 (dos veces) con un solo comando
`e0.csv`, `e1.csv`, `e2.csv`, `e2b.csv`	Evidencia de cada número de la tabla
`server_e0.log` a `server_e2b.log`	Logs del servidor en cada corrida
`README.md`	Instrucciones, tabla y respuestas
e2_1000tarjetas.csv	Evidencia de la pregunta 3: E2 con 1000 tarjetas distintas

- El servidor responde y `/config` informa los trabajadores activos y las entradas del caché.
- Hay una diferencia medible de throughput entre W=1 y W=8: 0.67 contra 24.99 req/s.
- El campo `desde_cache` distingue las dos rutas: 0 % en E0 y E1, y 94.4 % en E2.